Descend

A Safe GPU Systems Programming Language

Bastian Köpcke, Sergei Gorlatch, Michel Steuwer

University of Münster, Technische Universität Berlin

August 20, 2025

Computing on GPUs is great!

GPU Programming is too hard!

CUDA

```cuda
__global__ void transpose(const float *input, float *output) {
  __shared__ float tmp[1024];
  for (int j = 0; j < 32; j += 8) {
    tmp[threadIdx.y+j*32+threadIdx.x] =
      input[(blockIdx.y*32+threadIdx.y+j)*2048 + blockIdx.x*32+threadIdx.x];
  }
  __syncthreads();
  for (int j = 0; j < 32; j += 8) {
    output[(blockIdx.x*32+threadIdx.y+j)*2048 + blockIdx.y*32+threadIdx.x] =
      tmp[threadIdx.x*32+threadIdx.y+j];
  }
}
```

Kernel functions are executed by thousands of threads

CUDA

```
__global__ void transpose(const float *input, float *output) {
  __shared__ float tmp[1024];
  for (int j = 0; j < 32; j += 8) {
    tmp[threadIdx.y+j*32+threadIdx.x] =
      input[(blockIdx.y*32+threadIdx.y+j)*2048 + (blockIdx.x*32+threadIdx.x)];
  }
  __syncthreads();
  for (int j = 0; j < 32; j += 8) {
    output[(blockIdx.x*32+threadIdx.y+j)*2048 + blockIdx.y*32+threadIdx.x] =
      tmp[threadIdx.x*32+threadIdx.y+j];
  }
}
```

Kernel functions are executed by thousands of threads

Threads are hierarchically organized into multi-dimensional *blocks*
Blocks are hierarchically are organized into a multi-dimensional *grid*

CUDA

```
__global__ void transpose(const float *input, float *output) {
  __shared__ float tmp[1024];
  for (int j = 0; j < 32; j += 8) {
    tmp[threadIdx.y+j*32+threadIdx.x]
      input[(blockIdx.y*32+threadIdx.y+j)*2048 + blockIdx.x*32+threadIdx.x];
  }
  __syncthreads();
  for (int j = 0; j < 32; j += 8) {
    output[(blockIdx.x*32+threadIdx.y+j)*2048 + blockIdx.y*32+threadIdx.x] =
      tmp[threadIdx.x*32+threadIdx.y+j];
  }
}
```

explicit indexing to determine where a thread accesses memory

Kernel functions are executed by thousands of threads

Threads are hierarchically organized into multi-dimensional *blocks*
Blocks are hierarchically are organized into a multi-dimensional *grid*

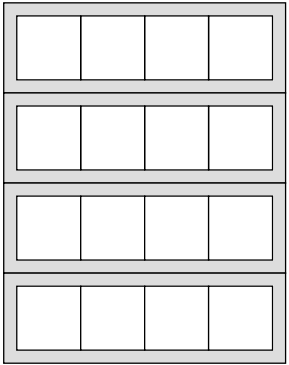
CUDA

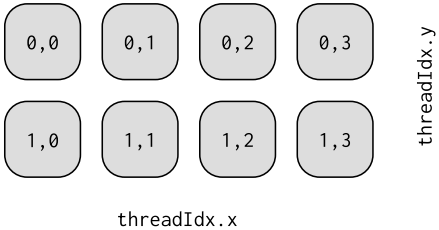
```
__global__ void transpose(const float *input, float *output) {
  __shared__ float tmp[1024];
  for (int j = 0; j < 32; j += 8) {
    tmp[threadIdx.y+j*32+threadIdx.x]
      input[(blockIdx.y*32+threadIdx.y+j)*2048 + blockIdx.x*32+threadIdx.x];
  }
  __syncthreads();
  for (int j = 0; j < 32; j += 8) {
    output[(blockIdx.x*32+threadIdx.y+j)*2048 + blockIdx.y*32+threadIdx.x] =
      tmp[threadIdx.x*32+threadIdx.y+j];
  }
}
```

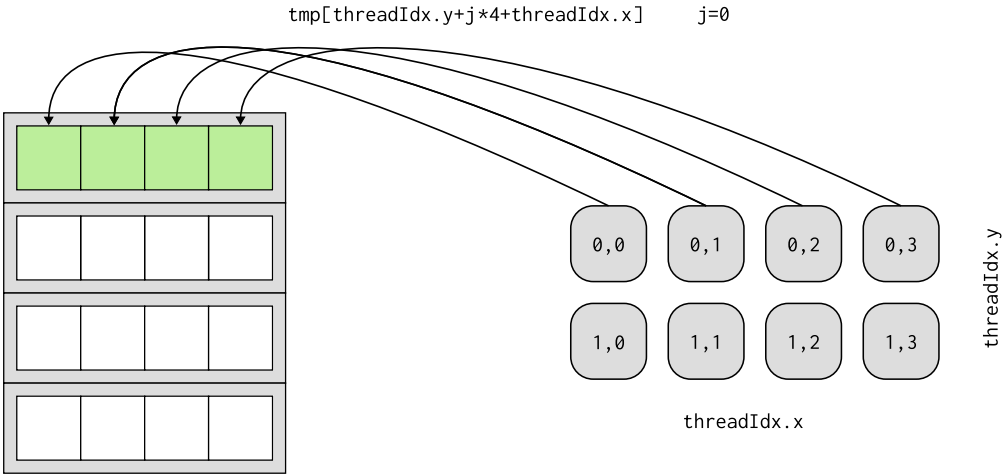
explicit indexing to determine where a thread accesses memory

Is the implementation correct?

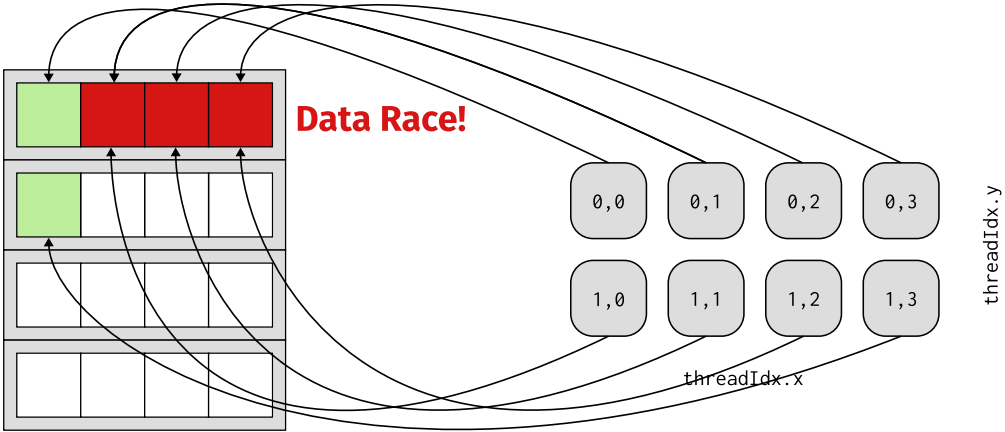
tmp[threadIdx.y+j*4+threadIdx.x] j=0





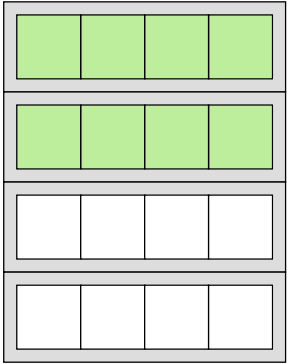

tmp[threadIdx.y+j*4+threadIdx.x]
j=0
0,0
0,1
0,2
0,3
1,0
1,1
1,2
1,3
threadIdx.y
threadIdx.x

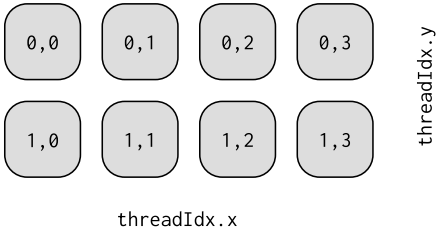
tmp[threadIdx.y+j*4+threadIdx.x] j=0



~~tmp[threadIdx.y+j*4+threadIdx.x]~~ j=0

tmp[(threadIdx.y+j)*4+threadIdx.x]





How can we create a safe GPU programming language?

Descend

A Safe GPU Systems Programming Language

"if the program successfully compiles,
then it is free of data races and other memory problems"

Descend

```
fn transpose(
  input: &gpu.global [[f32;2048];2048],
  output: &uniq gpu.global [[f32;2048];2048],
  [grid.blocks.forall(X).forall(Y)] tmp: &uniq gpu.shared [[f32;32];32]
) -[grid: gpu.grid<XY<64,64>,XY<32,8>>]-> () {
  sched(Y,X) block in grid {
    sched(Y,X) thread in block {
      for i in 0..4 {
        tmp.transpose.group_by_row::<32,4>[[thread]][i] =
          input.group_by_tile::<32,32>.transpose[[block]].group_by_row::<32,4>[[thread]][i]
      };

      for i in 0..4 {
        output.group_by_tile::<32,32>[[block]].group_by_row::<32,4>[[thread]][i]=
          tmp.group_by_row::<32,4>[[thread]][i]
      }
    }
  }
}
```

Descend

```
fn transpose(
  input: &gpu.global [[f32;2048];2048],
  output: &uniq gpu.global [[f32;2048];2048],
  [grid.blocks.forall(X).forall(Y)] tmp: &uniq gpu.shared [[f32;32];32]
) -[grid: gpu.grid<XY<64,64>,XY<32,8>>]-> () {
  sched(Y,X) block in grid {
    sched(Y,X) thread in block {
      for i in 0..4 {
        tmp.transpose.group_by_row::<32,4>[[thread]][i] =
          input.group_by_tile::<32,32>.transpose[[block]].group_by_row::<32,4>[[thread]][i]
      };

      for i in 0..4 {
        output.group_by_tile::<32,32>[[block]].group_by_row::<32,4>[[thread]][i]=
          tmp.group_by_row::<32,4>[[thread]][i]
      }
    }
  }
}
```

function is explicitly annotated with a 2D **grid** of 64 × 64 **blocks** with 32 × 8 **threads** each

Descend

```
fn transpose(
  input: &gpu.global [[f32;2048];2048],
  output: &uniq gpu.global [[f32;2048];2048],
  [grid.blocks.forall(X).forall(Y)] tmp: &uniq gpu.shared [[f32;32];32]
) -[grid: gpu.grid<XY<64,64>,XY<32,8>>]-> () {
  sched(Y,X) block in grid {
    sched(Y,X) thread in block {
      for i in 0..4 {
        tmp.transpose.group_by_row::<32,4>[[thread]][i] =
          input.group_by_tile::<32,32>.transpose[[block]].group_by_row::<32,4>[[thread]][i]
      };

      for i in 0..4 {
        output.group_by_tile::<32,32>[[block]].group_by_row::<32,4>[[thread]][i]=
          tmp.group_by_row::<32,4>[[thread]][i]
      }
    }
  }
}
```

collectively executed by the entire grid

Descend

```
fn transpose(
  input: &gpu.global [[f32;2048];2048],
  output: &uniq gpu.global [[f32;2048];2048],
  [grid.blocks.forall(X).forall(Y)] tmp: &uniq gpu.shared [[f32;32];32]
) -[grid: gpu.grid<XY<64,64>,XY<32,8>>]-> () {
  sched(Y,X) block in grid {
    sched(Y,X) thread in block {
      for i in 0..4 {
        tmp.transpose.group_by_row::<32,4>[[thread]][i] =
          input.group_by_tile::<32,32>.transpose[[block]].group_by_row::<32,4>[[thread]][i]
      };

      for i in 0..4 {
        output.group_by_tile::<32,32>[[block]].group_by_row::<32,4>[[thread]][i]=
          tmp.group_by_row::<32,4>[[thread]][i]
      }
    }
  }
}
```

collectively executed by each block in the grid

Descend

```
fn transpose(
  input: &gpu.global [[f32;2048];2048],
  output: &uniq gpu.global [[f32;2048];2048],
  [grid.blocks.forall(X).forall(Y)] tmp: &uniq gpu.shared [[f32;32];32]
) -[grid: gpu.grid<XY<64,64>,XY<32,8>>]-> () {
  sched(Y,X) block in grid {
    sched(Y,X) thread in block {
      for i in 0..4 {
        tmp.transpose.group_by_row::<32,4>[[thread]][i] =
          input.group_by_tile::<32,32>.transpose[[block]].group_by_row::<32,4>[[thread]][i]
      };

      for i in 0..4 {
        output.group_by_tile::<32,32>[[block]].group_by_row::<32,4>[[thread]][i]=
          tmp.group_by_row::<32,4>[[thread]][i]
      }
    }
  }
}
```

collectively executed by each thread in the block

Descend

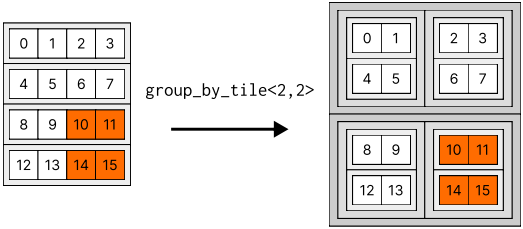
```
fn transpose(
  input: &gpu.global [[f32;2048];2048],
  output: &uniq gpu.global [[f32;2048];2048],
  [grid.blocks.forall(X).forall(Y)] tmp: &uniq gpu.shared [[f32;32];32]
) -[grid: gpu.grid<XY<64,64>,XY<32,8>>]-> () {
  sched(Y,X) block in grid {
    sched(Y,X) thread in block {
      for i in 0..4 {
        tmp.transpose.group_by_row::<32,4>[[thread]][i] =
          input.group_by_tile::<32,32>.transpose[[block]].group_by_row::<32,4>[[thread]][i]
      };

      for i in 0..4 {
        output.group_by_tile::<32,32>[[block]].group_by_row::<32,4>[[thread]][i] =
          tmp.group_by_row::<32,4>[[thread]][i]
      }
    }
  }
}
```

safely write in parallel over blocks and threads using **views**

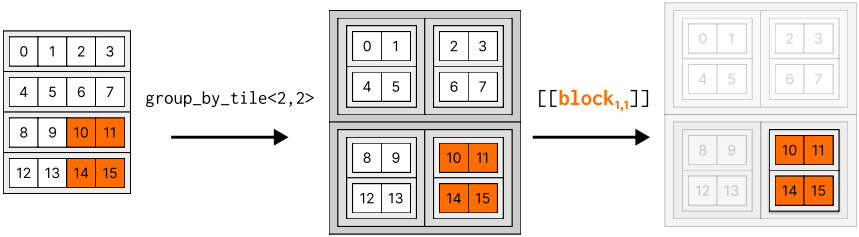
```
output.group_by_tile<2,2>[[block_{1,1}]].group_by_row<2,2>[[ thread_{0,1} ]]
```

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

```
output.group_by_tile<2,2>[[block_{1,1}]].group_by_row<2,2>[[ thread_{0,1} ]]
```

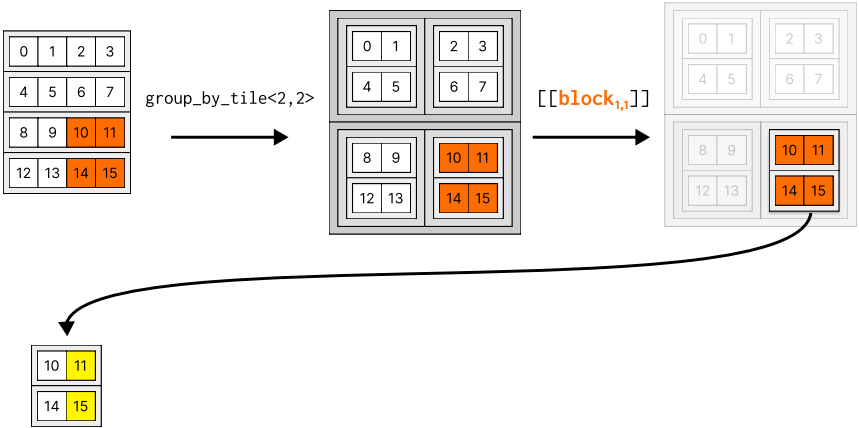


`output.group_by_tile<2,2>[[block`$_{1,1}$`]].group_by_row<2,2>[[ thread`$_{0,1}$]]



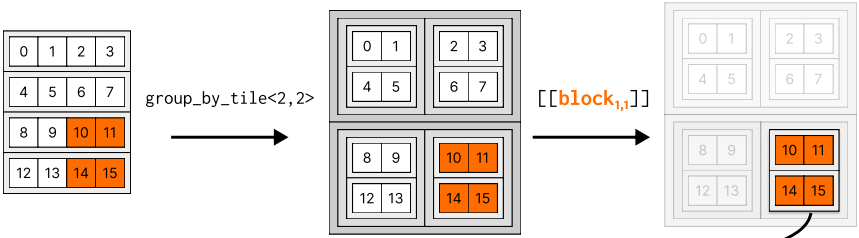
block$_{1,1}$ collectively owns this tile

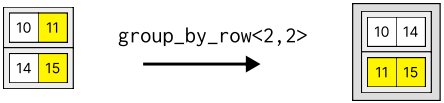
output.group_by_tile<2,2>[[block$_{1,1}$]].group_by_row<2,2>[[thread$_{0,1}$]]



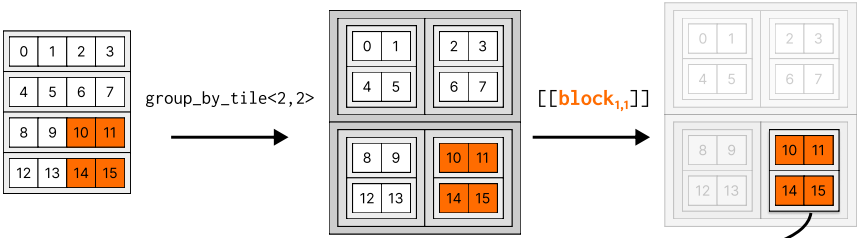
block$_{1,1}$ collectively owns this tile

output.group_by_tile<2,2>[[$\mathbf{block}_{1,1}$]].group_by_row<2,2>[[$\mathbf{thread}_{0,1}$]]

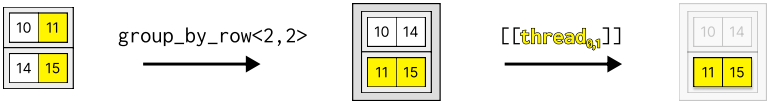




output.group_by_tile<2,2>[[block$_{1,1}$]].group_by_row<2,2>[[thread$_{0,1}$]]

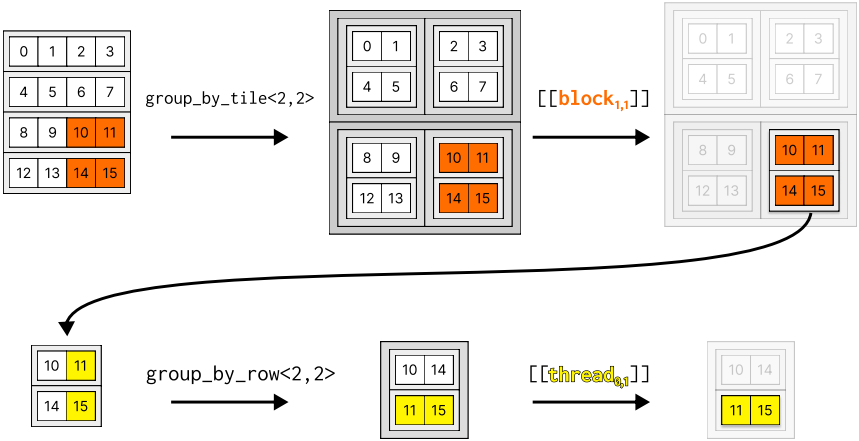


block$_{1,1}$ collectively owns this tile



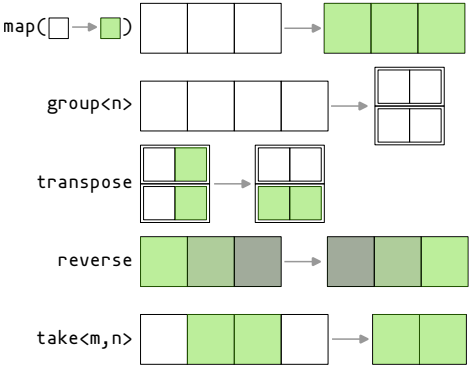
thread$_{0,1}$ owns these elements

`output.group_by_tile<2,2>[[block$_{1,1}$]].group_by_row<2,2>[[ thread$_{0,1}$ ]]`

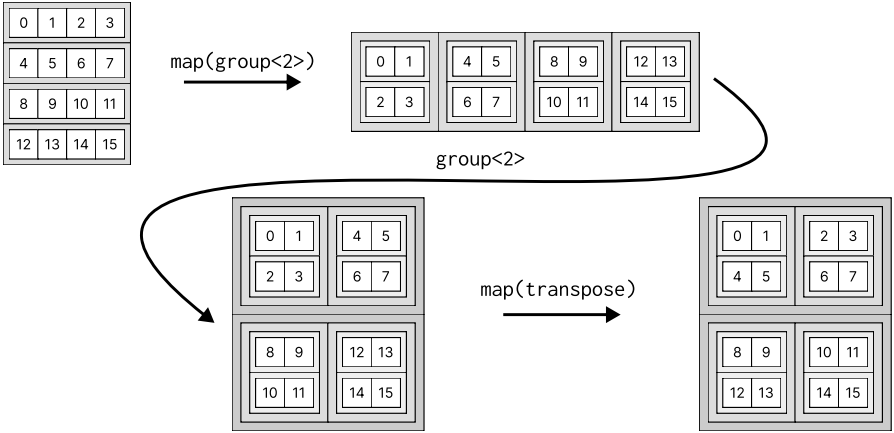


Descend's extended ownership model ensures that memory is accessed safely via views

Descend's five basic view primitives



```
group_by_tile<2,2> = map(group<2>).group<2>.map(transpose)
```



Views are composed to express complex memory access patterns

Descend

```
fn transpose(
  input: &gpu.global [[f32;2048];2048],
  output: &uniq gpu.global [[f32;2048];2048],
  [grid.blocks.forall(X).forall(Y)] tmp: &uniq gpu.shared [[f32;32];32]
) -[grid: gpu.grid<XY<64,64>,XY<32,8>>] -> () {
  sched(Y,X) block in grid {
    sched(Y,X) thread in block {
      for i in 0..4 {
        tmp.group_by_row::<32,4>[[thread]][i] =
          input.group_by_tile::<32,32>.transpose[[block]].group_by_row::<32,4>[[thread]][i]
      };

      for i in 0..4 {
        output.group_by_tile::<32,32>[[block]].group_by_row::<32,4>[[thread]][i] =
          tmp.transpose.group_by_row::<32,4>[[thread]][i]
      }
    }
  }
}
```

threads write safely into memory tmp

Descend

```
fn transpose(
  input: &gpu.global [[f32;2048];2048],
  output: &uniq gpu.global [[f32;2048];2048],
  [grid.blocks.forall(X).forall(Y)] tmp: &uniq gpu.shared [[f32;32];32]
) -[grid: gpu.grid<XY<64,64>,XY<32,8>>]-> () {
  sched(Y,X) block in grid {
    sched(Y,X) thread in block {
      for i in 0..4 {
        tmp.group_by_row::<32,4>[[thread]][i] =
          input.group_by_tile::<32,32>.transpose[[block]].group_by_row::<32,4>[[thread]][i]
      };

      for i in 0..4 {
        output.group_by_tile::<32,32>[[block]].group_by_row::<32,4>[[thread]][i] =
          tmp.transpose.group_by_row::<32,4>[[thread]][i]
      }
    }
  }
}
```

threads read safely from memory tmp

Descend

```
fn transpose(
  input: &gpu.global [[f32;2048];2048],
  output: &uniq gpu.global [[f32;2048];2048],
  [grid.blocks.forall(X).forall(Y)] tmp: &uniq gpu.shared [[f32;32];32]
) -[grid: gpu.grid<XY<64,64>,XY<32,8>>]-> () {
  sched(Y,X) block in grid {
    sched(Y,X) thread in block {
      for i in 0..4 {
        tmp.group_by_row::<32,4>[[thread]][i] =
          input.group_by_tile::<32,32>.transpose[[block]].group_by_row::<32,4>[[thread]][i]
      };

      for i in 0..4 {
        output.group_by_tile::<32,32>[[block]].group_by_row::<32,4>[[thread]][i] =
          tmp.transpose.group_by_row::<32,4>[[thread]][i]
      }
    }
  }
}
```

~~threads read safely from memory tmp~~

Descend

```
fn transpose(
  input: &gpu.global [[f32;2048];2048],
  output: &uniq gpu.global [[f32;2048];2048],
  [grid.blocks.forall(X).forall(Y)] tmp: &uniq gpu.shared [[f32;32];32]
) -[grid: gpu.grid<XY<64,64>,XY<32,8>>]-> () {
  sched(Y,X) block in grid {
    sched
      for
        t

      };

      for
        o

      }
    }
  }
}
```

missing synchronization leads to an error

Descend

```
error: conflicting memory access
  | line 15: tmp.transpose.group_by_row::<32,4>[[thread]][i]
  |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  | line 9: tmp.group_by_row::<32,4>[[thread]][i]
  |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Descend

```
fn transpose(
  input: &gpu.global [[f32;2048];2048],
  output: &uniq gpu.global [[f32;2048];2048],
  [grid.blocks.forall(X).forall(Y)] tmp: &uniq gpu.shared [[f32;32];32]
) -[grid: gpu.grid<XY<64,64>,XY<32,8>>]-> () {
  sched(Y,X) block in grid {
    sched(Y,X) thread in block {
      for i in 0..4 {
        tmp.group_by_row::<32,4>[[thread]][i] =
          input.group_by_tile::<32,32>.transpose[[block]].group_by_row::<32,4>[[thread]][i]
      };
      sync(block);
      for i in 0..4 {
        output.group_by_tile::<32,32>[[block]].group_by_row::<32,4>[[thread]][i] =
          tmp.transpose.group_by_row::<32,4>[[thread]][i]
      }
    }
  }
}
```

explicit barrier synchronization on the entire block

Descend

```
fn transpose(
  input: &gpu.global [[f32;2048];2048],
  output: &uniq gpu.global [[f32;2048];2048],
  [grid.blocks.forall(X).forall(Y)] tmp: &uniq gpu.shared [[f32;32];32]
) -[grid: gpu.grid<XY<64,64>,XY<32,8>>]-> () {
  sched(Y,X) block in grid {
    sched(Y,X) thread in block {
      for i in 0..4 {
        tmp.group_by_row::<32,4>[[thread]][i] =
          input.group_by_tile::<32,32>.transpose[[block]].group_by_row::<32,4>[[thread]][i]
      };
      sync(block);
      for i in 0..4 {
        output.group_by_tile::<32,32>[[block]].group_by_row::<32,4>[[thread]][i] =
          tmp.transpose.group_by_row::<32,4>[[thread]][i]
      }
    }
  }
}
```

Descend ensures that there are not data races

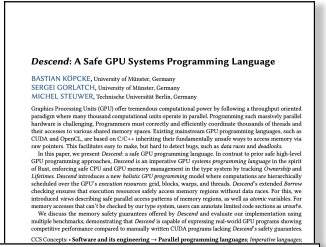
Current State of Descend

Further guarantees provided by Descend:

- Respect separation between CPU & different GPU memories
- Assumptions between CPU and GPU code match
- Safe memory accesses in branching code
- Absence of barrier divergence

What we did:

- Formalized the type system
- Implemented compiler to CUDA



***Descend*: A Safe GPU Systems Programming Language**

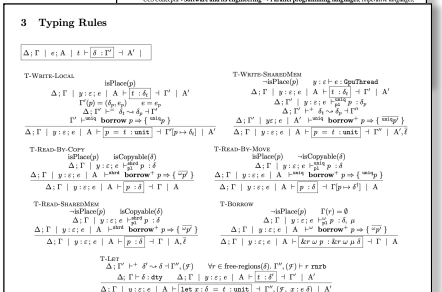
BASTIAN KÖPCKE, University of Münster, Germany
SERGEI GORLATCH, University of Münster, Germany
MICHEL STEUWER, Technische Universität Berlin, Germany

Graphics Processing Units (GPU) offer tremendous computational power by following a throughput oriented paradigm where many thousand computational units operate in parallel. Programming such massively parallel hardware is challenging. Programmers must correctly and efficiently coordinate thousands of threads and their accesses to various shared memory spaces. Existing mainstream GPU programming languages, such as CUDA and OpenCL, are based on C/C++ inherenting their fundamentally unsafe ways to access memory via raw pointers. This facilitates easy to make, but hard to detect bugs, such as data *races* and *deadlocks*.

In this paper, we present *Descend*: a safe GPU programming language. In contrast to prior safe high-level GPU programming approaches, *Descend* is an imperative *GPU systems programming* language in the spirit of Rust, enforcing safe CPU and GPU memory management in the type system by tracking *Ownership* and *Lifetimes*. *Descend* introduces a new *holistic GPU programming* model where computations are hierarchically scheduled over the GPU's execution resources: grid, blocks, warps, and threads. *Descend*'s extended *Borrow* checking ensures that execution resources safely access memory regions without data races. For this, we introduced views describing safe parallel access patterns of memory regions, as well as atomic variables. For memory accesses that can't be checked by our type system, users can annotate limited code sections as *unsafe*.

We discuss the memory safety guarantees offered by *Descend* and evaluate our implementation using multiple benchmarks, demonstrating that *Descend* is capable of expressing real-world GPU programs showing competitive performance compared to manually written CUDA programs lacking *Descend*'s safety guarantees.

CCS Concepts • **Software and its engineering → Parallel programming languages**. *Imperative languages;*



3 Typing Rules

$$\boxed{\Delta;\Gamma \mid e;\mathbb{A} \mid t \vdash \overline{\boxed{\delta:\Gamma'}}} \dashv \mathbb{A}' \mid$$

T-Write-Local

$$\frac{\begin{array}{c}\Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash \boxed{t:\delta_l} \dashv \Gamma' \mid \mathbb{A}' \\ \Gamma'(p) = (\delta_{pl}, \eta_p) \quad e \approx e_p \\ \Delta;\Gamma' \vdash^{\sigma} \delta_l \rightsquigarrow \delta_g : \Gamma'' \\ \Gamma'' \vdash^{\mathbb{A}\setminus\mathbb{A}} \mathbf{borrow}\ p \Rightarrow \{\ ^{\mathbb{A}\setminus\mathbb{A}}\eta\ \}\end{array}}{\Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash \boxed{p = t : \mathtt{unit}} \dashv \Gamma'[p \mapsto \delta_l] \mid \mathbb{A}'}$$

$\mathrm{isPlace}(p)$ $\quad \neg\mathrm{isPlace}(p)$

T-Write-SharedMem

$$\frac{\begin{array}{c}\Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash \boxed{t:\delta_l} \dashv \Gamma' \mid \mathbb{A}' \\ \Delta;\Gamma' \mid y;e;e \vdash^{\mathbb{A}\setminus\mathbb{A}} p : \delta_p \\ \Delta;\Gamma' \vdash^{\sigma} \delta_l \rightsquigarrow \delta_g : \Gamma'' \\ \Gamma'' \vdash^{\mathbb{A}\setminus\mathbb{A}} \mathbf{borrow}\ p \Rightarrow \{\ ^{\mathbb{A}\setminus\mathbb{A}}\eta\ \}\end{array}}{\Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash \boxed{p = t : \mathtt{unit}} \dashv \Gamma'' \mid \mathbb{A}', \vec{t}}$$

$y : t \vdash e : \mathtt{GpuThread}$

T-Read-By-Copy

$$\frac{\begin{array}{c}\Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash^{\mathbb{A}\setminus\mathbb{A}} p : \delta \\ \Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash^{\mathbb{A}\setminus\mathbb{A}} \mathbf{borrow}\ p \Rightarrow \{\ \overline{\sim\rho'}\ \}\end{array}}{\Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash \boxed{p : \delta} \dashv \Gamma \mid \mathbb{A}}$$

$\mathrm{isPlace}(p)$ $\quad \mathrm{isCopyable}(\delta)$

T-Read-By-Move

$$\frac{\begin{array}{c}\Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash^{\mathbb{A}\setminus\mathbb{A}} p : \delta \\ \Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash^{\mathbb{A}\setminus\mathbb{A}} \mathbf{borrow}\ p \Rightarrow \{\ ^{\mathbb{A}\setminus\mathbb{A}}\eta\ \}\end{array}}{\Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash \boxed{p : \delta} \dashv \Gamma[p \mapsto \delta'] \mid \mathbb{A}}$$

$\mathrm{isPlace}(p)$ $\quad \neg\mathrm{isCopyable}(\delta)$

T-Read-SharedMem

$$\frac{\begin{array}{c}\Delta;\Gamma \mid y;e;e \vdash^{\mathbb{A}\setminus\mathbb{A}} p : \delta \\ \Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash^{\mathbb{A}\setminus\mathbb{A}} \mathbf{borrow}\ p \Rightarrow \{\ \overline{\sim\rho'}\ \}\end{array}}{\Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash \boxed{p : \delta} \dashv \Gamma \mid \mathbb{A}, \vec{t}}$$

$\mathrm{isPlace}(p)$ $\quad \neg\mathrm{isCopyable}(\delta)$

T-Borrow

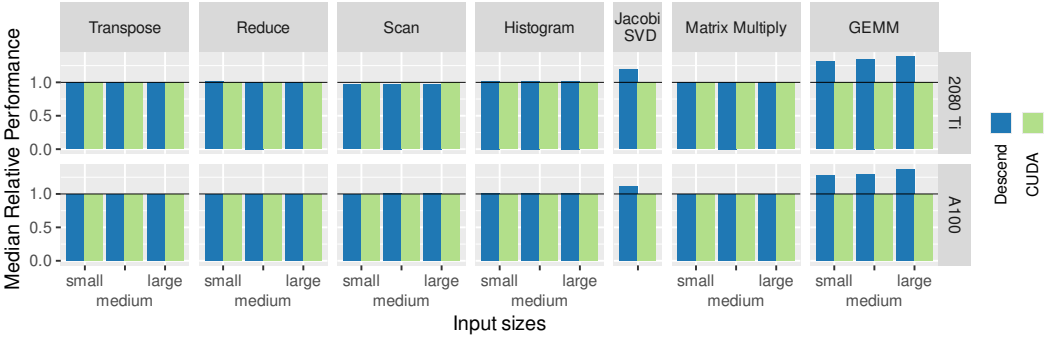
$$\frac{\begin{array}{c}\Delta;\Gamma \mid y;e;e \vdash^{\omega} p : \delta, \rho \\ \Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash^{\omega} \mathbf{borrow}\ p \Rightarrow \{\ \overline{\sim\rho'}\ \}\end{array}}{\Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash \boxed{\&r\,\omega\,p : \&r\,\omega\,\rho\,\delta} \dashv \Gamma \mid \mathbb{A}}$$

$\mathrm{isPlace}(p)$ $\quad \Gamma(r) = \emptyset$

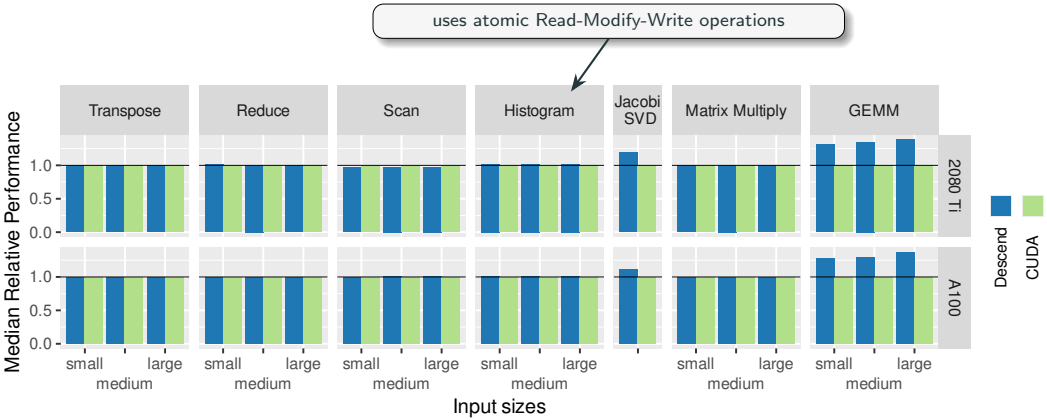
T-Let

$$\frac{\begin{array}{c}\Delta;\Gamma' \vdash^{\omega} \delta' \rightsquigarrow \delta : \Gamma'', (\mathcal{F}) \qquad \forall r \in \text{free-regions}(\delta), \Gamma'', (\mathcal{F}) \vdash r\ \mathrm{nrb} \\ \Delta;\Gamma \vdash^{\sigma} \delta : \mathtt{dty} \qquad \Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash \boxed{t:\delta'} \dashv \Gamma' \mid \mathbb{A}'\end{array}}{\Delta;\Gamma \mid y;e;e \mid \mathbb{A} \vdash \boxed{\mathtt{let}\ x : \delta = t : \mathtt{unit}} \dashv \Gamma'', (\mathcal{F}, x : e\ \delta) \mid \mathbb{A}'}$$

Evaluation

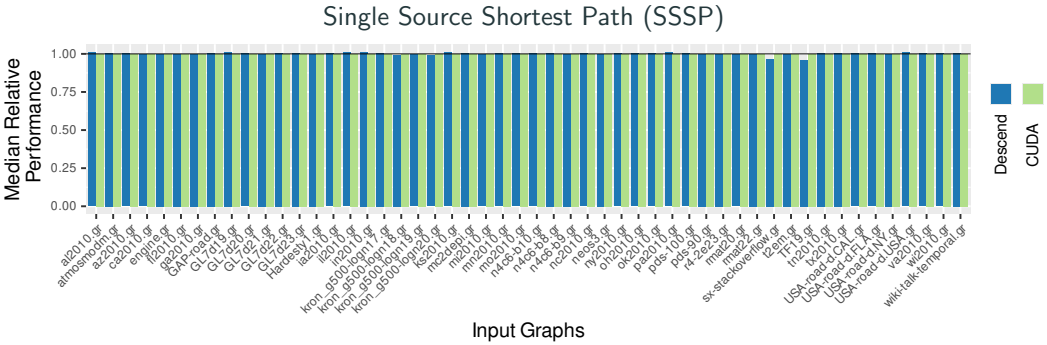


Programs generated from Descend achieve competitive performance to handwritten CUDA.



Programs generated from Descend achieve competitive performance to handwritten CUDA.

unsafe: Interfacing with CUDA



unsafe code provides a quick method of interfacing with existing CUDA code

Conclusion

- *Descend* assists programmers in managing CPU and GPU memory and enforcing previously implicit assumptions
- *Descend* extends the concept of ownership to the thread hierarchy and uses views to ensure safe memory accesses on the GPU
- The evaluation shows that *Descend* is expressive enough to write programs that achieve performance on-par with handwritten CUDA

Website for Descend: `https://descend-lang.org`

bastian.koepcke@uni-muenster.de